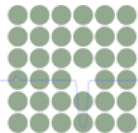




UNIVERSITÄT
LEIPZIG



Paper Presentation and Implementation

Single-shot Self-supervised Particle Detection in Microscopy

October 10, 2025

Edgar Harutyunyan

Molecular Nanophotonics Group
Peter Debye Institute for Soft Matter Physics
Universität Leipzig

Outline

1. LodeSTAR:
Self-supervised Object Detection
2. Status Update
3. Results & Discussion

LodeSTAR: Self-supervised Object Detection



Single-shot self-supervised object detection in microscopy

Received: 14 March 2022

Accepted: 15 November 2022

Published online: 05 December 2022

Benjamin Midtvedt¹, Jesús Pineda ¹, Fredrik Skärberg¹, Erik Olsén ²,
Harshith Bachimanchi ¹, Emelie Wesén³, Elin K. Esbjörner ³, Erik Selander ⁴,
Fredrik Höök ², Daniel Midtvedt¹ & Giovanni Volpe ¹ 

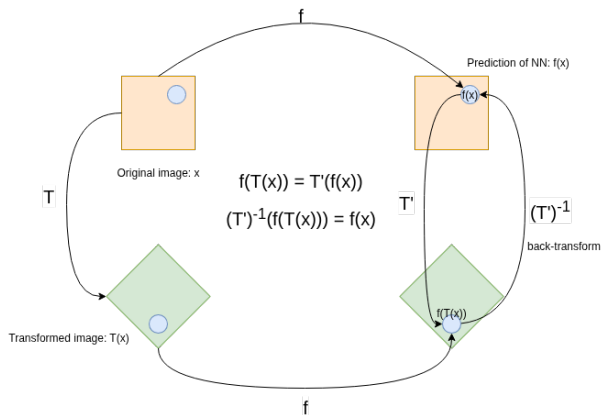
What is LodeSTAR?

- **LodeSTAR:** Localization and **d**etection from **S**ymmetries, **T**ranslations, And **R**otations
- **Goal:** Detect objects in microscopy images without manual annotations
- **Key Innovation:** Self-supervised learning using data transformations
- **Output:** Dense predictions of object positions and confidence weights

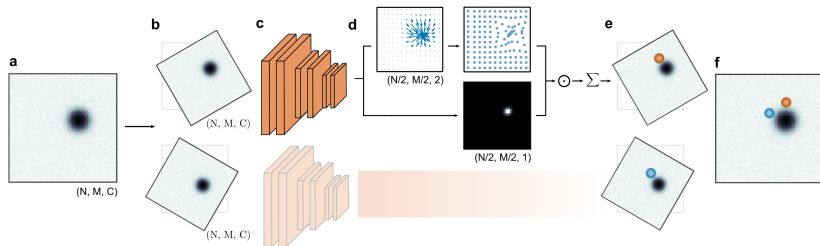
Core Concept: Self-supervised Learning

- **No manual labels required**
- **Data transformations** create supervision signal
- **Consistency across transformations** = learning objective
- **Translation equivariance** ensures robust detection
- **Process:**
 - Transform input image
 - Train model to predict consistent outputs
 - Use consistency as supervision signal

Geometric self-distillation: Equivariance principle



Training



- **Data Augmentation:** Apply k transformations to input image
- **CNN Processing:** Process transformed images through CNN
- **Parameter Optimization:** Optimize parameters to minimize consistency loss

LodeSTAR Network Architecture

Backbone:

- **Input:** 1-channel grayscale image
- **3× Conv2D (3×3, 32) + ReLU**
- **MaxPool2D (2×2)** after 3rd layer
- **8× Conv2D (3×3, 32) + ReLU**
- **Conv2D (1×1, 3)** → outputs: Δx , Δy , ρ

Core Design Features:

- Uniform 32-channel depth
- Translation-equivariant
- Dense prediction at every pixel
- Lightweight: $\sim 100\text{K}$ parameters
- No fully connected layers

Architecture Summary Table

Layer(s)	Type	Channels	Kernel
0–2	Conv2D + ReLU	$1 \rightarrow 32$	3×3
3	MaxPool2D	–	2×2
4–11	Conv2D + ReLU	$32 \rightarrow 32$	3×3
12	Conv2D	$32 \rightarrow 3$	1×1

Purpose: Regress vector field $(\Delta x, \Delta y)$ and confidence ρ from input image with minimal architecture.

Loss Functions

Within-image Consistency:

$$L_{\text{within}} = \|y_{\text{pred}} - \bar{y}\| \cdot w \quad (1)$$

$$\bar{y} = \sum w_i y_i / \sum w_i \quad (2)$$

- L_{within} : Enforces agreement between all pixel-wise predictions and the consensus (mean) location
- Encourages confident regions to dominate the final prediction

Between-image Equivariance:

$$L_{\text{between}} = \|y_{\text{pred}}^T - T(\bar{y})\| \quad (3)$$

- L_{between} : Ensures predictions remain consistent under transformations
- Encourages the network to learn transformation-equivariant representations

Total: $L_{\text{total}} = L_{\text{within}} + L_{\text{between}}$

Output Processing & Detection

Output Components:

- **Delta X**: X-displacement field ($H/2 \times W/2$)
- **Delta Y**: Y-displacement field ($H/2 \times W/2$)
- **Weights**: Confidence scores ($H/2 \times W/2$)
- **Sigmoid** applied to weights (0-1 range)
- **Coordinate grids** for absolute positions

Detection Process:

1. **Forward pass** through trained model
2. **Extract** δ_x, δ_y , weights
3. **Apply sigmoid** to weights
4. **Generate coordinate grids**
5. **Calculate absolute positions**
6. **Find local maxima** in confidence map
7. **Apply threshold** for final detections

Status Update

Project Overview

- **Goal:** Implement and evaluate LodeSTAR for particle detection
- **Scope:** Single-particle detection on synthetic microscopy images
- **Particle Types:** Janus, Ring, Spot, Ellipse, Rod
- **Implementation:** PyTorch + DeepTrack + PyTorch Lightning
- **Tracking:** Logger and Weights & Biases (WandB) for experiment monitoring

Data Generation Pipeline

Image Generation:

- **Synthetic particles** with realistic parameters
- **Multiple datasets** for different scenarios:
 - Same shape, same size
 - Same shape, different size
 - Different shape, same size
 - Different shape, different size
- **XML annotations** with bounding boxes and SNR values

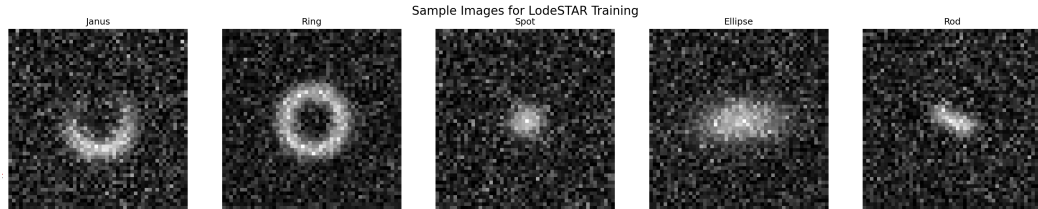
Technical Details:

- **Image size:** 416×416 pixels
- **SNR range:** 1-30
- **Intensity range:** 0.1-1.0
- **Particle parameters:**
Type-specific values
- **100 images** per dataset type

Sample Images Generation

Sample Images:

- **Clean samples** for each particle type
- **50×50 pixels** for training
- **High SNR (10)** for clear features
- **Realistic parameters** for each particle type
- **Stored in** data/Samples/



Training Pipeline Implementation

Training Setup:

- **Single-particle models** for each particle type
- **Data augmentation:** Random multiplication and addition
- **400 samples per epoch** (configurable)
- **200 epochs** maximum training
- **Batch size:** 8 (balanced for memory)

Technical Stack:

- **PyTorch Lightning** for training
- **DeepTrack** for data pipelines
- **WandB** for experiment tracking
- **Mixed precision** training (16-bit)
- **GPU acceleration** when available

Training Configuration & Optimization

Parameter	Value
Learning Rate	0.0001
Batch Size	8
Max Epochs	200
N Transforms	4
Alpha	1.0
Beta	0.0
Cutoff	0.8

Experiment 192 Results: 6-cycle debugging optimized parameters

- **Default LodeSTAR** architecture performs best (F1: 0.526)
- **Final configuration:** 50px images, n_transforms=4, only intensity augmentation

Testing Pipeline & Results

Testing Setup:

- **Load trained models** from checkpoints
- **Test on generated datasets** in `data/Testing/`
- **Parse XML annotations** for ground truth
- **Calculate detection metrics:** Precision, Recall, F1
- **Distance threshold:** 20 pixels for positive detection

Training Success:

- **All 5 models** completed 200 epochs
- **Loss convergence** observed
- **Checkpoints saved** every epoch
- **WandB logging** active
- **Model weights** exported

Results & Discussion

Current Status & Key Results

Completed:

- **All 5 models** successfully trained (200 epochs each)
- **Comprehensive evaluation** across 4 dataset types
- **Architecture debugging** completed (Experiment 192)
- **Performance analysis** with statistical metrics

Key Findings:

- **Default LodeSTAR** architecture performs best
- **High recall** on same shape scenarios (0.89-0.97)
- **Moderate performance** on different shapes
- **Optimized parameters** identified

Performance Results Summary

Dataset Type	Avg F1	Avg Precision	Avg Recall	Best Model
SSSS	0.561	0.638	0.919	Ring (0.800)
SSDS	0.394	0.488	0.889	Rod (0.524)
DSSS	0.254	0.534	0.439	Janus (0.273)
DSDS	0.261	0.399	0.625	Spot (0.397)

Key Insights:

- **Best performance** on same shape scenarios (expected)
- **Ring model** excels on same shape (F1: 0.800)
- **Rod model** shows good generalization
- **Lower performance** on different shapes (expected)

Model Performance Summary

Top Performers:

- **Ring:** F1: 0.800 (same shape)
- **Rod:** F1: 0.524 (good generalization)
- **Spot:** F1: 0.670 (consistent)
- **Janus:** F1: 0.469 (moderate)
- **Ellipse:** F1: 0.150 (struggles)

Performance Patterns:

- **Same shape:** High recall (0.89-0.97)
- **Different shape:** Lower recall (0.44-0.63)
- **Precision varies:** 0.24-0.89 across scenarios
- **Shape-specific** performance differences

Key Findings & Future Work

Key Achievements:

- **Self-supervised learning** works for particle detection
- **High recall** on same shape scenarios (0.89-0.97)
- **Default LodeSTAR** architecture performs best
- **Complete pipeline** implemented successfully

Areas for Improvement:

- **Low precision** on different shapes
- **High false positive rates**
- **Shape-specific training** needed
- **Parameter optimization** required